CSE 4224

Digital System Design Laboratory

Graph Explorer

BFS/DFS Hardware Path Search on Basys 3 (Verilog / Vivado)

Implemented by

Amit Paul (2007083)

Khandoker Abid Hasan (2007107)

Date: _____

1 Project description

The **Graph Explorer** is a digital system that runs on the Digilent **Basys 3** board (Xilinx Artix-7). The user encodes a small directed graph (four nodes) on switches, selects a start node, a target node, a traversal mode (breadth-first or depth-first search), and whether to report **minimum** or **maximum** hop count along valid paths. The design then executes the chosen algorithm in hardware and shows the result on the seven-segment display, with LEDs and an RGB LED for status and visited feedback. The RTL is written in **Verilog** and organized as a top module, a core `graph_engine`, memory for the graph and algorithmic structures, a central control finite-state machine, and peripheral blocks for debouncing, clock division, and display.

2 Project structure

The design follows a clear hierarchy from simulation to silicon-oriented top level:

1. **Testbench** (`tb_graph_explorer`) instantiates the hardware top and drives clocks, switches, and the center button for simulation.
2. **Top RTL** (`graph_explorer_top`) connects board pins to internal logic: debounced button input, divided tick for multiplexed displays, the `graph_engine` datapath, and seven-segment / RGB outputs.
3. **Graph engine** (`graph_engine`) wires together the `register_set` (latched configuration), `memory_unit` (adjacency, queue, stack), `control_unit` (load sequence and BFS/DFS execution), and `graph_alu` (simple combinational support).

Data generally flows from switches and button events into the registers and control unit; the control unit reads and writes `memory_unit` (edges, BFS queue, DFS stack) and updates display and LED outputs through the top-level glue.

3 Project units

All RTL lives under `DSD_Project.srcs/sources_1/new/`. The subsections below **describe every unit** in the project. Section 4 then gives a **slightly longer** focus on the two largest behavioral blocks which are completed (`memory_unit.v`, `control_unit.v`).

3.1 Descriptions of all units

`graph_explorer_top` **(`graph_explorer_top.v`).** This is the synthesizable top: it maps board pins (switches, center button, LEDs, seven-segment anodes/cathodes, RGB) to internal wires, instantiates the debouncer and edge detector for the button, the clock divider tick, the `graph_engine`, and the display/status drivers. It is the module targeted for Basys 3 bitstream generation.

`debouncer` **(`debouncer.v`).** Filters mechanical bounce on the center button by requiring a stable raw level for many clock cycles before updating a debounced output. That clean level is what downstream logic treats as the "real" button state.

`edge_detector` **(`edge_detector.v`).** Watches the debounced button and emits a **single-cycle pulse** when a press edge is detected. The control unit uses that pulse as "user confirmed this step" without counting multiple transitions per physical press.

Table 1: Design units and implementation files.

Unit / module	**Role**	**File(s)**
graph_explorer_top	Top-level integration: I/O, debounce chain, clock divider, engine, segment and RGB drivers.	graph_explorer_top.v
debouncer	Suppresses switch bounce on the center button before edge detection.	debouncer.v
edge_detector	Produces a one-cycle pulse from a stable button level.	edge_detector.v
clk_divider	Generates a low-frequency tick for time-multiplexed seven-segment scanning.	clk_divider.v
graph_engine	Structural wrapper connecting registers, memory, control, and ALU.	graph_engine.v
register_set	Holds latched graph word, start/target indices, BFS/DFS flag, and min/max mode.	register_set.v
memory_unit	Stores 4×4 adjacency, BFS FIFO, and DFS stack; services edge queries.	memory_unit.v
control_unit	Main FSM: multi-step configuration, BFS shortest-hop search, DFS path enumeration for min/max length.	control_unit.v
graph_alu	Small combinational block (documentation-oriented ALU slice).	graph_alu.v
seg7_driver	Active-low segment patterns and anode multiplexing for four digits.	seg7_driver.v
rgb_status	Maps a 2-bit status code to RGB LED drive.	rgb_status.v
Shared definitions	Optional parameters / constants for the graph core.	graph_defs.vh
Testbench	Simulation harness for the top module.	tb_graph_explorer.v
Constraints	Basys 3 pin and timing constraints for implementation.	basys3_pins.xdc

`clk_divider` (`clk_divider.v`). Divides the board clock to produce a slow **tick** used to time-multiplex the four seven-segment digits so only one anode is active at a time while the eye integrates a stable image.

`graph_engine` (`graph_engine.v`). A **structural** (non-behavioral) wrapper: it declares the interconnection between `register_set`, `memory_unit`, `control_unit`, and `graph_alu`, matching the datapath-plus-controller partition used in the lab report.

`register_set` (`register_set.v`). Holds configuration loaded from switches across the stepped workflow: the 16-bit adjacency word, start and target node indices, a flag for BFS versus DFS, and a flag for minimum versus maximum hop objective. Write enables come from the control unit on each configuration phase.

`memory_unit` (`memory_unit.v`). Central storage for the graph and algorithmic working data: the adjacency bit vector, a FIFO for BFS frontier nodes, and a stack for the iterative DFS path enumerator, plus combinational edge readout for indices (i, j). (See Section 4 for a concise summary of how it supports BFS/DFS.)

`control_unit` (`control_unit.v`). The main **finite-state machine**: multi-phase loading from switches, then either BFS distance search or DFS-based enumeration for min/max path length, while driving the memory ports and updating displays and status. (Section 4 highlights the same file in more detail.)

`graph_alu` (`graph_alu.v`). A small **combinational** block (increment/compare style) kept as a separate module so the course block diagram explicitly includes an "ALU" slice next to the register file, even though much arithmetic also appears inside the control FSM.

`seg7_driver` (`seg7_driver.v`). Translates four BCD or hex nibbles from the control path into active-low segment patterns and sequences the anodes using the slow tick so four digits share one cathode bus.

`rgb_status` (`rgb_status.v`). Decodes a 2-bit status from the control unit into on/off patterns for the board's red, green, and blue LEDs to show idle, busy, success, or error-style states at a glance.

Shared definitions (`graph_defs.vh`). Optional Verilog header for shared parameters or constants (graph size, widths) so multiple `.v` files can stay consistent if constants are centralized.

Testbench (`tb_graph_explorer.v`). Simulation-only Verilog under `DSD_Project.srcs/sim_1/new/`: generates the clock, applies switch and button stimulus (often via tasks), and instantiates `graph_explorer_top` so behavior can be verified in the simulator without hardware.

Constraints (`basys3_pins.xdc`). Xilinx constraints file under `DSD_Project.srcs/constrs_1/new/`: maps logical ports to Basys 3 package pins and supplies basic timing constraints for implementation.

4 Brief description of the memory unit and control unit

These two modules contain most of the **algorithmic** RTL. Both are plain Verilog sources (no separate behavioral models).

4.1 Memory unit — `memory_unit.v`

Source file: `memory_unit.v` (`DSD_Project.srcs/sources_1/new/`).

The memory unit stores the **directed** 4×4 adjacency as sixteen bits and answers **edge queries** used while expanding neighbors. For BFS it provides a **circular queue** (push, pop, empty, full) of node IDs. For DFS-style search over **simple paths** it provides a **stack** of packed records (node, neighbor index k, visited bitmask, depth). The **st_poke** interface updates the top entry without popping so a parent frame can increment k after trying a child—essential for iterative DFS in hardware. **clr_qs** clears queue and stack for a new run while **adj_we** reloads the graph from the register file when the user commits the switch pattern.

4.2 Control unit — `control_unit.v`

Source file: `control_unit.v` (`DSD_Project.srcs/sources_1/new/`).

The control unit implements the **user protocol**: after reset, successive button pulses latch graph, start, target, and mode (BFS/DFS and min/max) into `register_set` and into `memory_unit` for the adjacency. It then enters **run** substates: for BFS, clear/init, enqueue start, dequeue/expand/test-target/enqueue neighbors with distance bookkeeping; for DFS, clear stack, push a start frame, then loop with stack peek, target test, neighbor advance (**poke** + optional child **push**), and pop when all neighbors are tried. It drives `memory_unit.v` ports (`q_*`, `st_*`, `edge_i`/`edge_j`, `adj_we`, `clr_qs`) and updates **seven-segment** digits and **RGB** status for each phase and final outcome.

5 Discussion

Partitioning the design into `memory_unit` and `control_unit` keeps **storage and connectivity** separate from the **long FSM** that implements both setup and two different graph algorithms. That split matches common practice in course-style datapath plus controller architectures and makes simulation easier: memory behavior (FIFO/stack) can be reasoned about independently of high-level BFS/DFS flow.

The Basys 3 offers limited human input (switches and one useful button), which led to a **stepped configuration workflow** rather than a full keyboard or UART interface. Representing a 4×4 adjacency on sixteen switches is workable for a lab demo but bounds graph size; the same architecture can be extended later with more storage and a wider control loop.

Simulation with `tb_graph_explorer.v` validates clocking and basic interaction without hardware. **Synthesis and on-board** testing remain important next steps to confirm timing, pin mapping (`basys3_pins.xdc`), and real switch/button behavior against the ideal model.

6 Future work

- Complete the whole project
- Run full **Vivado implementation** (synthesis, place-and-route), close timing at the chosen clock, and record **LUT/FF/BRAM** usage for the final report.
- Exercise the design on **physical hardware**, tune debounce intervals if needed, and capture a short **demo checklist** (graph patterns, expected displays).
- Extend the testbench with **self-checking** cases (e.g., disconnected graph, single-edge path, start equals target) and longer runs to observe end-to-end waveforms.

- Explore **larger** N or multi-cycle graph loading (e.g., UART or multi-button flow) if course scope allows.
- Optional **UX** improvements: clearer seven-segment legends, single-step debug, or a written appendix formalizing BFS distance versus DFS simple-path enumeration.